

Veritas — AI-Driven Misinformation & Fake News Detection System

Project Document

1. What Is the Project?

Veritas is a web-based platform that evaluates the credibility of news articles and produces a transparent, explainable credibility assessment rather than a simple "Real" or "Fake" label.

The system runs every submitted article through a three-stage AI pipeline:

1. **Claim Extraction** — identifies the discrete factual claims made in the article.
2. **Cross Verification** — assesses whether each claim is corroborated, contradicted, or untracked.
3. **Semantic Analysis** — detects bias, propaganda, logical fallacies, emotional manipulation, and context stripping in the article as a whole.

These three stages feed into a documented **ensemble scoring engine** that combines claim-level verification with semantic signals and the source's authority tier to produce a **Credibility Score (0–100)** and a final verdict of **True**, **Suspect**, or **False** — along with a full breakdown of the evidence behind that verdict.

Veritas supports two usage modes:

- **Single Article Detection** — analyze one article at a time through a web form, with results shown as an interactive dossier (credibility gauge, verdict stamp, collapsible claim cards).
- **Bulk Detection (CSV Upload)** — analyze an entire dataset of articles in one pass, with accuracy, precision, recall, F1, and a confusion matrix computed automatically whenever ground-truth labels are supplied.

It is built as a Flask REST API with a modular, blueprint-based backend, which means the same detection endpoints that power the web dashboard can also be called directly by other applications — browser extensions, CMS platforms, or external services — making Veritas usable both as a standalone tool and as an embeddable service.

Analysis can run in three interchangeable engine modes: a fully **offline, rule-based engine** that requires no API key and no internet access, or one of two **LLM-backed engines** (NVIDIA's [openai/gpt-oss-20b](#) or Anthropic Claude) for deeper language understanding, with automatic fallback to the offline engine if an API call fails.

2. Project Observation

The motivation for building Veritas came from a set of observed weaknesses in how misinformation spreads and how existing detection tools respond to it.

Observation 1 — Information spreads faster than fact-checking

News, social posts, and blog content now reach large audiences within minutes of publication. Manual fact-checking is thorough but slow — by the time a claim is formally investigated, misleading content has often already been widely shared. A detection layer that can produce an immediate, evidence-based signal at the point of reading has real value, even if it is not a substitute for professional fact-checkers.

Observation 2 — Modern misinformation rarely consists of outright lies

Misleading articles today are more often built from a mixture of:

- genuine but selectively presented facts,
- emotionally loaded language,
- unattributed or vaguely attributed claims,
- statistics stripped of context,
- rhetorical patterns (loaded framing, calls to "share before it's deleted," appeals to distrust of institutions).

This means detection cannot rely purely on whether a sentence is factually false — it also has to account for *how* the information is framed and *who* is making the claim.

Observation 3 — Reputation-only and keyword-only detection breaks down quickly

Systems that rely solely on domain blacklists or keyword filters run into predictable failure modes:

- A generally reliable outlet can still publish an inaccurate or poorly sourced story.
- New unreliable sources appear faster than blacklists can be updated.
- A misleading article can quote or repost content that originated from a credible source.
- Some misinformation is contextual (a true quote used to imply a false conclusion) rather than factually incorrect on its face.

This pointed toward a design that treats source reputation as *one input among several* — an authority-weighting factor — rather than the sole basis for a verdict.

Observation 4 — Binary labels without explanation undermine trust

A "Fake" or "Real" label with no supporting reasoning gives a user no way to evaluate whether the tool's judgment is sound, and no way to learn what to look for themselves next time. Any credible detection system needs to show its work: which claims it found, how it evaluated them, what rhetorical patterns it flagged, and how the final score was calculated.

3. Problem Statement

The digital information ecosystem is increasingly exposed to misinformation that blends real facts with misleading framing, unverifiable attribution, and emotionally manipulative language. Existing detection approaches — static blacklists, keyword filters, and opaque binary classifiers — struggle to keep pace with new sources, fail to account for contextual manipulation, and rarely explain their reasoning to the end user.

There is a need for a detection system that:

- extracts and evaluates individual factual claims rather than judging an article as a single undivided block of text,
- accounts for source authority without treating it as the only signal,
- surfaces rhetorical and linguistic manipulation techniques explicitly,
- produces a graded credibility score instead of a binary label,
- shows the evidence and calculation behind every verdict, and
- is available both as a usable interface and as an integrable API, so it can support real-time decision-making rather than only after-the-fact review.

Veritas was built to address this gap using a defined, auditable pipeline and a documented scoring formula rather than an opaque model.

4. Why Was This Project Built?

Veritas was built to give people a fast, transparent, and reproducible way to assess an article's credibility, with four specific goals:

Improve public awareness. Give readers a structured breakdown of a claim's plausibility before they decide to trust or share it, rather than asking them to take a label on faith.

Support human fact-checkers. Surface the claims, attribution gaps, and rhetorical red flags in an article automatically, so a human reviewer can focus their time on the claims that actually need deeper investigation.

Increase transparency in AI-assisted moderation. Every score Veritas produces is traceable back to a specific formula, specific claim-level verification results, and specific semantic flags — nothing is a black-box probability with no explanation attached.

Provide a foundation that scales. By exposing the pipeline as a REST API with both single-article and bulk-CSV modes, Veritas is usable interactively by one person or dropped into an automated workflow that needs to screen large batches of content.

5. How the System Works

Step 1 — Input Collection

Veritas accepts two forms of input, by design:

- A single article, submitted with **title, body text, source name, and topic** through the web form or the `/api/single/analyze` endpoint.
- A **CSV dataset** of articles (`article_id, title, text, source, topic, ground_truth`), submitted through the bulk upload interface or the `/api/bulk/analyze` endpoint.

There is deliberately no URL-fetching, PDF, or DOCX ingestion in the current version — all analysis works directly from text the user provides, keeping the system's inputs explicit and auditable.

Step 2 — Stage A: Claim Extraction

The article is broken into candidate sentences, and 3–10 factual claims are selected based on how "claim-worthy" each sentence is — sentences containing statistics, named entities, direct attribution ("according to..."), years, or research-related language are prioritized. Each claim is returned with:

- a claim ID,
- the claim statement itself,
- its alleged source (extracted from attribution language where present),
- a verification category (statistical claim, direct quote/attribution, scientific/research claim, historical/event claim, or general factual claim).

Step 3 — Stage B: Cross Verification

Each extracted claim is assessed and classified as:

- **Corroborated** — language patterns and attribution are consistent with verifiable reporting,
- **Contradicted** — the claim shows hedging or unverifiable framing inconsistent with confirmed reporting,
- **No Tracked Records** — no strong signal either way.

Each classification includes an evidence summary, a domain authority estimate, and a confidence value.

Step 4 — Stage C: Semantic Analysis

The article as a whole is analyzed for five manipulation patterns — bias, propaganda framing, logical fallacies, emotional manipulation, and context stripping — and assigned a **Consistency Score (1–5)** that reflects how neutral versus manipulative the writing is overall.

Step 5 — Ensemble Scoring

Stages B and C are combined into a per-claim score:

$$S_c = (W \times 0.7) + (L / 5 \times 0.3)$$

where **W** is 1.0 for Corroborated, 0.5 for No Tracked Records, and 0 for Contradicted, and **L** is the Stage C consistency score. The mean of all per-claim scores is then weighted by the source's **authority multiplier** (Tier 1 = 1.00, Tier 2 = 0.85, Tier 3 = 0.50) to produce the final:

$$CS_A = \text{mean}(S_c) \times \text{AuthorityMultiplier} \times 100$$

A score of 70+ is voted **True**, 40–69 is voted **Suspect**, and below 40 is voted **False**. The system also reports **Model Disagreement** — how much the individual claim scores diverge from one another — as an additional signal of how consistent the evidence is internally.

Step 6 — Engine Selection

All three stages can run through one of three interchangeable engines:

- **Offline (rule-based)** — deterministic linguistic heuristics, no external calls, always available.
- **NVIDIA** ([openai/gpt-oss-20b](#) via NVIDIA's hosted API) — LLM-driven extraction, verification, and semantic analysis.
- **Anthropic Claude** — the same three stages performed via the Claude API.

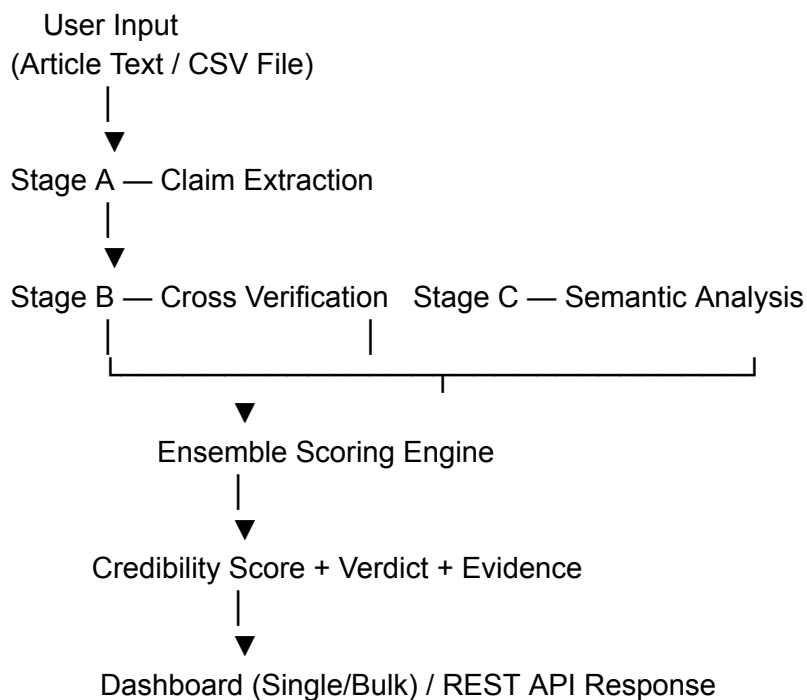
If an LLM call fails for any reason, the pipeline automatically falls back to the offline engine so a request never hard-fails.

Step 7 — Output

Single Article mode returns the credibility score, verdict, source tier, model disagreement, the full claim-by-claim breakdown, and the semantic analysis flags, rendered as an interactive dossier.

Bulk mode returns a results table for every row, a downloadable results CSV ([article_id](#), [title](#), [topic](#), [source](#), [credibility_score](#), [ensemble_vote](#), [claim_count](#), [model_disagreement](#), [ground_truth](#), [match](#)), and — when [ground_truth](#) is present — accuracy, macro precision/recall/F1, and a full confusion matrix.

System Architecture



6. How to Use the System

Veritas is designed for both interactive use and programmatic integration.

As a web application:

- Enter an article's title, body, source, and topic in the Single Article form to get an instant credibility dossier.
- Upload a CSV of multiple articles in Bulk Detection mode to screen a dataset at once, and download the annotated results.

As an API:

- `POST /api/single/analyze` — analyze one article and receive the full pipeline output as JSON.
- `POST /api/bulk/analyze` — upload a CSV and receive per-row results plus metrics as JSON, along with a downloadable results file.
- `GET /api/bulk/download/<filename>` — retrieve a completed bulk run's results CSV.
- `GET /api/health` — basic service health check.

This API-first design means Veritas can be embedded into a browser extension, a CMS moderation queue, or any other application that needs a credibility signal, without requiring the caller to use the web dashboard at all.

7. How to Maintain the System

Recommended ongoing maintenance practices:

- **Review and expand the source-tier lists** (`modules/utils.py`) periodically, since new outlets and domains appear faster than any static list can anticipate.
 - **Refine the offline heuristics** (hedging language, propaganda patterns, fallacy patterns) as new rhetorical techniques in misinformation are observed.
 - **Monitor LLM engine reliability** — track how often NVIDIA/Claude calls fail and fall back to offline mode, and adjust timeouts or retry logic if failure rates rise.
 - **Re-validate the ensemble formula against labeled data** — periodically run the bulk pipeline against a labeled dataset and check that accuracy/precision/recall/F1 remain within acceptable ranges; adjust the scoring weights if drift is detected.
 - **Rotate and secure API keys** — ensure NVIDIA/Claude keys are stored only in environment variables (`.env`), never committed to source control, and rotated periodically.
 - **Keep dependencies current** — update Flask, Werkzeug, and `requests` regularly to pick up security patches.
 - **Clean up generated bulk-result files** in `uploads/` periodically if running on a host with persistent storage, since each bulk run writes a new results CSV.
-

8. How to Extend the System

Veritas is intentionally modular — each pipeline stage is an independent module (`claim_extractor.py`, `corroboration.py`, `semantic_analysis.py`, `ensemble_engine.py`) — so it can be extended in stages rather than requiring a full rewrite. Planned and possible extensions include:

- **Retrieval-Augmented Generation (RAG) verification** — replacing or supplementing the current heuristic/LLM-based Stage B with real-time retrieval against fact-checking databases, government publications, and academic sources for genuine cross-referencing rather than plausibility judgment alone.
- **Ensemble machine learning classifiers** — adding trained models (e.g., TF-IDF + Logistic Regression, or fine-tuned transformer models such as BERT/RoBERTa/DeBERTa) as additional voices in the ensemble, alongside the current rule-based and LLM engines.
- **Graph Neural Network source analysis** — modeling source publishing history, citation relationships, and content propagation patterns to detect coordinated disinformation campaigns rather than evaluating each article in isolation.
- **Deeper Explainable AI reporting** — expanding the current evidence summaries into a fuller explanation report with confidence intervals and sentence-level highlighting.

- **Multilingual support** — extending claim extraction and semantic analysis beyond English.
- **URL and multimedia ingestion** — adding optional article-URL fetching and image/deepfake analysis as separate, clearly-flagged input modes.
- **Real-time social media monitoring** — streaming ingestion for continuous credibility scoring rather than on-demand submission only.
- **Persistent case history and dashboards** — storing past analyses per user for trend tracking, rather than the current stateless request/response model.

Because each stage returns a well-defined JSON structure, any of these extensions can be added by replacing or augmenting a single module without changing the API contract the rest of the system depends on.

9. Conclusion

Veritas addresses the core weaknesses observed in conventional fake news detection — slow fact-checking relative to how fast content spreads, sophistication in modern misinformation techniques, the brittleness of reputation-only detection, and the lack of explanation in binary classifiers — by combining claim-level extraction, cross-verification, semantic/rhetorical analysis, and a documented ensemble formula into a single transparent pipeline. Its modular, API-first architecture and interchangeable offline/LLM engines make it usable as a standalone dashboard today, while leaving a clear path toward more advanced techniques such as RAG-based verification, ensemble transformer models, and graph-based source analysis as the system matures.